

Point-in-Time Multi-Agent LLM Trading: A Reproducible LangGraph Stack with Regime-Conditioned Evaluation and Fair Backtesting

Rohit Edward Martires

Independent researcher, Goa, India

`rohitmartires14@gmail.com`

29 May 2026

Abstract

Large language models are increasingly used in financial decision pipelines, yet published agent stacks are often hard to reproduce and rarely evaluated under distinct market conditions on historical as-of dates. We present **Hindsight 20/20**, an open **LangGraph** stack with structured **Pydantic** outputs, point-in-time data access, optional **ticker anonymization**, and a fee-aware **paper backtest**. We evaluate the **full** pipeline on **RELIANCE.NS** and **TCS.NS** in **exogenous bear and bull windows** (per-ticker underlying return at most -10% or at least +10%), each from \$100,000 paper cash. In **both bear windows**, the agent loses less than buy-and-hold; in **bull windows**, outcomes diverge (RELIANCE lags B&H; TCS nearly tracks it). Frozen CSV artifacts reproduce tables and figures. Code and artifacts: <https://github.com/RMartires/hindsight>

Keywords: algorithmic trading; large language models; multi-agent systems; reproducibil-

ity; point-in-time data; market regimes.

0.1 Introduction

Researchers are experimenting with LLMs inside trading workflows, but rarely report how the **same stack** behaves across **different underlying trends** or **more than one issuer**. **Hindsight 20/20** extends the open-source **TradingAgents** multi-agent LangGraph framework (Xiao et al., 2024) with point-in-time data access, optional ticker anonymization, and a reproducible backtest harness; we evaluate this extended stack under exogenous bear and bull windows. We evaluate the **full** pipeline on **RELIANCE.NS** and **TCS.NS** in four independent runs (bear and bull per ticker). The implementation, frozen evaluation artifacts, and figure scripts are available at <https://github.com/RMartires/hindsight>. Section 2 surveys related work; Section 3 the system; Section 4 results; Section 5 concludes; Section 6 lists contributions.

0.2 Related Work

LLM trading agents motivate multi-role graphs (Xiao et al., 2024; Yu et al., 2023; Yang et al., 2023). We emphasize reproducible engineering: one codepath, schema-constrained outputs, temporal controls, CSV artifacts.

Point-in-time data and leakage require explicit simulation horizons and optional anonymization.

Regime-split evaluation on the same implementation is uncommon; we report four frozen runs with **per-ticker calendars**, not regime prediction.

Execution realism via a paper ledger with stated fees.

0.3 Method / System

TradingAgentsGraph composes analysts, bull/bear **debate roles** (internal agents, distinct from market regimes), trader, and risk. Core analyst → debate → trader → risk routing follows **TradingAgents** (Xiao et al., 2024); our modifications focus on temporal data policy, schema-constrained outputs, and the backtest/evaluation pipeline described below. Section 4 uses **PAPER_ABLATION=full**. **backtest_mvp.py --dates-csv** produces wide schedule CSVs.

0.3.1 Ticker anonymization

Large language models trained on public text often encode firm-specific narratives—brand reputation, sector themes, and historical events—tied to ticker symbols and company names. In a point-in-time backtest, that **pretraining leakage** can bias decisions: the same vendor data may elicit different agent behavior on **RELIANCE.NS** versus an unseen alias. Hind-sight 20/20 optionally **anonymizes the issuer** for all LLM-facing prompts and tool outputs: each real symbol maps to a deterministic synthetic id (**STOCK_####**, hash-derived); tool wrappers **de-anonymize** before market-data API calls, then **re-scrub** responses; news headlines additionally pass through a lightweight proper-noun scrubber. Frozen E1 runs use **enable_anonymization=True** (default in **backtest_mvp.py**). We do not ablate anonymization in Section 4—reporting with it **on** is a conservative choice that reduces name-based bias when comparing regimes and tickers.

0.3.2 Backtest metrics

The paper ledger records daily agent equity after fees. We report **total return** on agent equity; **maximum drawdown (MDD)** as peak-to-trough decline on the agent equity curve, with **buy-and-hold MDD** on the same underlying adjusted closes for comparison; and **Sharpe ratio** as mean daily simple return divided by its standard deviation, scaled by

$\sqrt{252}$ ¹. **No risk-free rate is subtracted** in Frozen E1 (excess Sharpe is future work). Short windows (67–111 trading days) yield noisy Sharpe estimates for discrete-signal agents.

0.4 Experiments

0.4.1 Regime definition

For each ticker, calendar windows in **regime-timeranges** are labeled **bear** if underlying buy-and-hold return is at most **-10%**, **bull** if at least **+10%** over the processed window. Each run starts from **\$100,000** fresh cash. Windows differ by ticker.

0.4.2 Protocol and results

Table 1. Four frozen runs. Sharpe is computed on **agent** daily equity returns (annualized; see footnote in Section 3.4; **no risk-free rate** subtracted).

Ticker	Regime	Window	Days	B&H	Agent	B&H MDD	Ag. MDD	Sharpe	Buy/Hold/Sell
RELIANCE	Bear	2024-08-01 - 2025-01-03	111	-17.4%	-7.5%	21.0%	5.1%	-3.12	7/28/76
RELIANCE	Bull	2025-01-01 - 2025-06-03	108	+15.1%	+5.6%	11.0%	2.8%	-3.35	9/25/74
TCS	Bear	2024-12-02 - 2025-04-03	88	-20.4%	-4.8%	23.9%	6.8%	-1.83	12/24/52
TCS	Bull	2024-06-03 - 2024-09-03	67	+21.9%	+19.6%	5.5%	1.7%	+4.25	13/14/40

Bear (2/2): agent loses less than buy-and-hold on **total return** and shows **lower path drawdown** (e.g., RELIANCE bear: agent MDD 5.1% vs B&H MDD 21.0%), consistent with SELL-heavy signals and reduced long exposure. **Bull:** RELIANCE lags B&H; TCS nearly tracks B&H. **Sharpe** is negative in three of four runs—including RELIANCE bull (+5.6% return)—because daily equity volatility dominates on short windows; only TCS bull is Sharpe-positive (+4.25). Interpret Sharpe alongside total return and MDD, not in isolation.

¹Annualization assumes 252 trading days per year (US equity convention). NSE calendars are typically ≈ 240 –250 trading days; we retain 252 for comparability with standard reported Sharpe ratios.

0.4.3 Figures

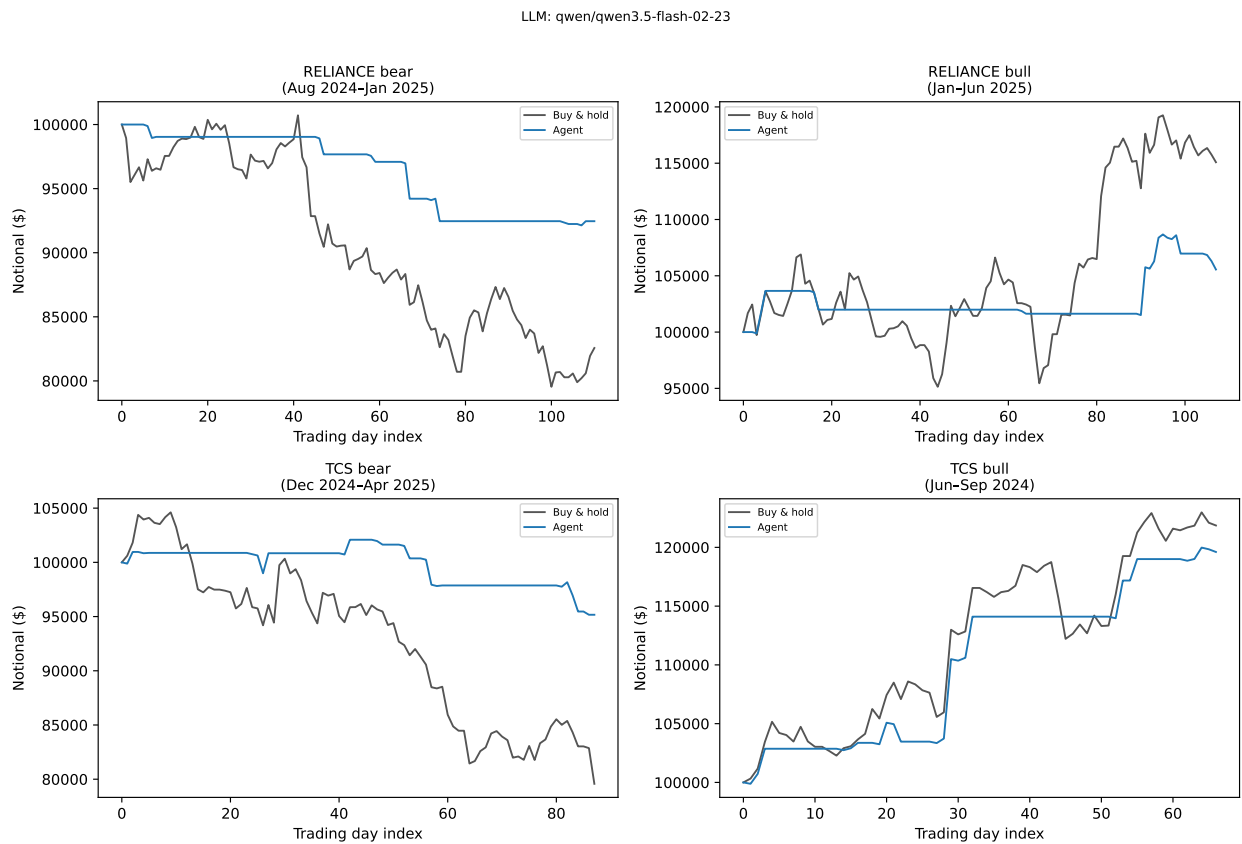


Figure 1: Figure 1. Agent vs buy-and-hold equity (2x2 grid).

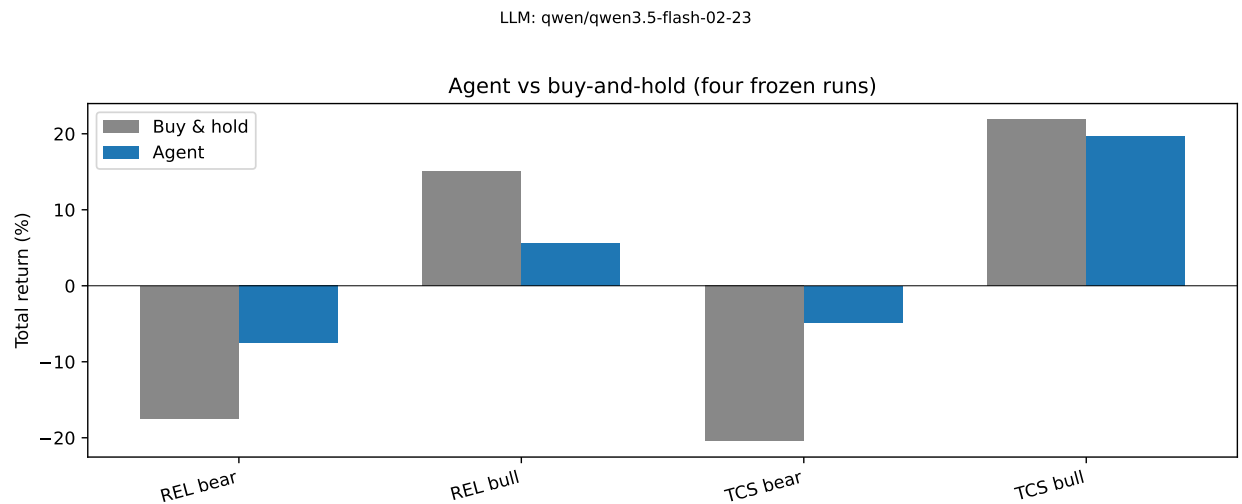


Figure 2: Figure 2. Total returns (four runs).

LLM: qwen/qwen3.5-flash-02-23

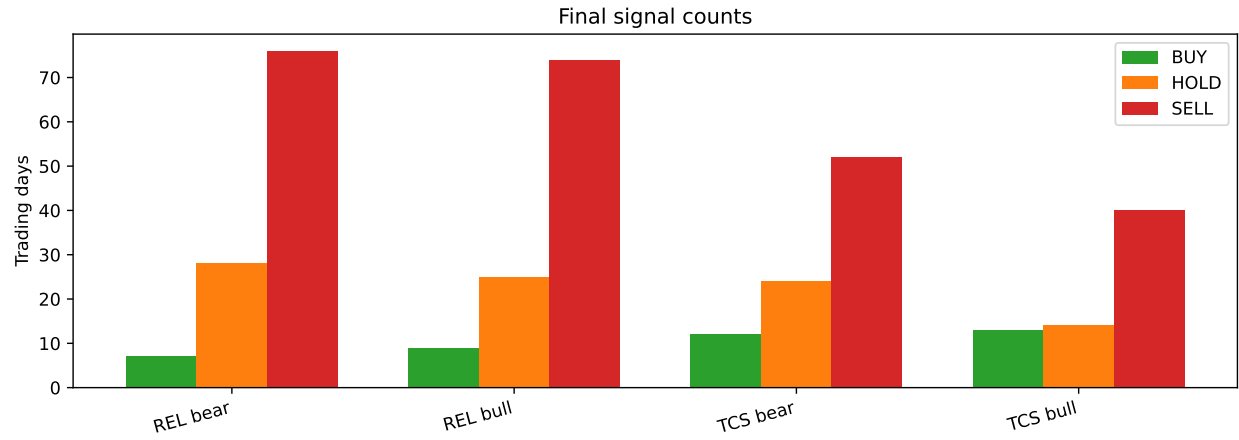


Figure 3: Figure 3. Signal counts.

LLM: qwen/qwen3.5-flash-02-23

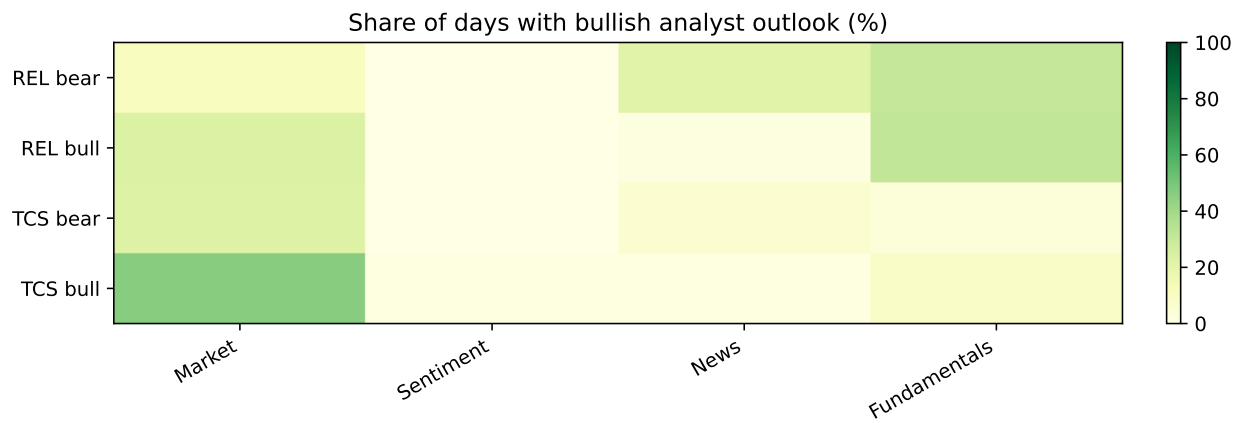


Figure 4: Figure 4. Bullish analyst-outlook share.

0.5 Conclusion

We presented an open LangGraph stack and **multi-ticker regime-conditioned** evaluation. Bear windows show relative downside protection on both **return and drawdown**; bull outcomes are heterogeneous. **Limitations:** two tickers, per-ticker calendars, one LLM; anonymization enabled but not ablated; Sharpe without a risk-free benchmark and on short windows.

0.6 Contributions

1. Multi-agent decision graph.
2. Multi-ticker regime-conditioned evaluation (four frozen CSVs).
3. Structured stage outputs.
4. Point-in-time policy and anonymization.
5. Backtest harness and analysis artifacts.

Acknowledgements

The agent graph implementation builds on **TradingAgents** (Xiao et al., 2024; Tauric Research, Apache License 2.0). We thank the authors for open-sourcing the framework.

References

- Xiao, Y., Sun, E., Luo, D. and Wang, W., 2024. TradingAgents: Multi-Agents LLM Financial Trading Framework. *arXiv preprint* arXiv:2412.20138 [q-fin.TR]. Available at: <https://arxiv.org/abs/2412.20138> (Accessed: 29 May 2026).
- Yang, H., Liu, X.-Y. and Wang, C.D., 2023. FinGPT: Open-Source Financial Large Language Models. *arXiv preprint* arXiv:2306.06031 [q-fin.ST]. Available at: <https://arxiv.org/abs/2306.06031> (Accessed: 29 May 2026).

Yu, Y., Li, H., Chen, Z., Jiang, Y., Li, Y., Zhang, D., Liu, R., Suchow, J.W. and Khashanah, K., 2023. FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design. *arXiv preprint* arXiv:2311.13743 [q-fin.CP]. Available at: <https://arxiv.org/abs/2311.13743> (Accessed: 29 May 2026).